实验3 计数器

HITSZ 实验与创新实践教育中心
Education Center of Experiments and Innovations, HITSZ

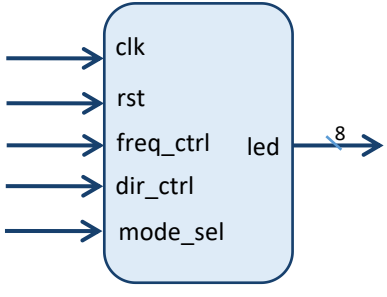
实验目的

（1）熟悉时序电路的开发与调试；

（2）掌握计数器电路的设计方法及计数器有关的电路应用。

实验内容

- 基于计数器，实现**时间间隔可调、方向可调**的8位硬件流水灯

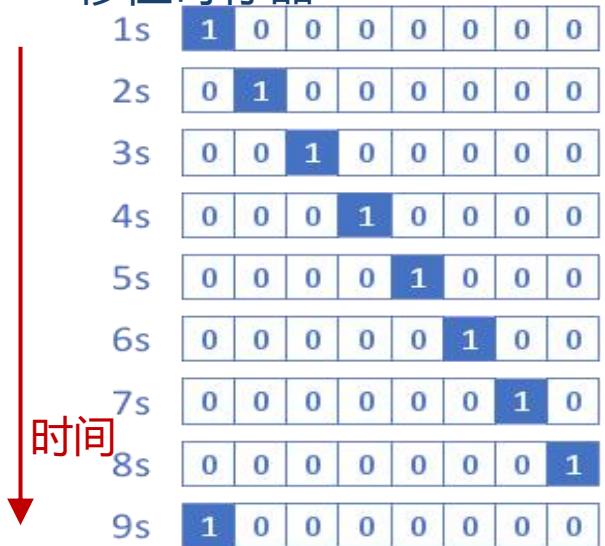




信号名	方向	位宽	描述
clk	input	1	时钟信号（Y18引脚，板载100MHz）
rst	input	1	异步高电平复位（S1按键开关，复位时只亮LED0，复位后流水灯停止）
freq_ctrl	input	1	频率设置（SW[0]拨码开关，间隔在0.002s、0.1s切换）
dir_ctrl	input	1	方向设置（SW[1]拨码开关，左移右移方向控制）
mode_sel	input	1	模式设置（1个灯流动、2个灯流动）
led	output	8	流水灯信号（LED[7:0]）

设计思路

- 每隔时间T，寄存器向右(或向左)**移动**1位
  - 计时信号生成：**计数器**实现**分频**，100MHz -> (1/T)Hz
  - 让8位的寄存器以一定时间间隔进行移位 —— 移位寄存器



实验原理-计数器实现模板


```
assign cnt_end = /* 结束条件 */;      // 计数结束条件

assign cnt_inc = /* 加1条件 */;       // 计数器自增条件
```

或always块描述

```
always @ (posedge clk or posedge rst) begin
    if (rst)            cnt <= 4'h0;          // 计数器位宽为4
    else if (cnt_end)   cnt <= 4'h0;          // 判断何时结束
    else if (cnt_inc)   cnt <= cnt + 4'h1;    // 判断是否继续加1
end

always @ (posedge clk or posedge rst) begin
    // 根据计数器的值产生相应的输出
end
```

实验原理-计数器实例

```
module counter(
    input  wire       clk,
    input  wire       rst,
    input  wire       button,
    output reg  [3:0] cnt
);

    wire cnt_end = cnt_inc & (cnt == 4'h9);     // 计数结束条件 (计数开始后，到9停止计数)

    reg  cnt_inc;                               // 计数器自增条件
    always @ (posedge clk or posedge rst) begin
        if (rst)          cnt_inc <= 1'h0;
        else if (button)  cnt_inc <= 1'h1;      // button按下开始计数
        else if (cnt_end) cnt_inc <= 1'h0;
    end

    always @ (posedge clk or posedge rst) begin
        if (rst)           cnt <= 4'h0;         // 计数器位宽为4
        else if (cnt_end)  cnt <= 4'h0;         // 判断何时结束
        else if (cnt_inc)  cnt <= cnt + 4'h1;   // 判断是否继续加1
    end

endmodule
```

- 分频后的信号应作为控制信号，而不是作为时钟来驱动其他模块

```
always @ (posedge clk or
                posedge rst)
begin
    if (rst)            ...;
    else if (cnt_end)   ...;
    else                ...;
end
```

循环移位：拼接符

- 拼接操作符用大括号{}来表示，将{}内的信号拼接成一个新的信号
- 循环右移：将信号的最低位移动到最高位，高7位往低位移动1位

```
reg [7:0] led;
always @ (posedge clk) begin
    led <= {led[0], led[7:1]};
end
```

- 循环左移：与循环右移相反

本实验相关的代码规范

① 正确的信号初始化：复位时赋初值

- 只能用于仿真的初始化：

  initial语句、 reg [7:0] a = 8'h0;

```
always @(posedge clk or posedge rst) begin
    if (rst) begin
        ......      // 给reg型变量赋初值
    end else begin
        ......      // 其他逻辑
    end
end
```

② 一个reg型变量，只能在一个always语句块中被赋值，否则多驱动

```
always @(posedge clk) begin
    if (flag1)      reg_abc <= value1;
    else if (flag2) reg_abc <= value2;
    else            reg_abc <= value3;
end
```

√ 正确的代码

```
always @(posedge clk) begin
    if (flag1) reg_abc <= value1;
end

always @(posedge clk) begin
    if (flag2) reg_abc <= value2;
    else       reg_abc <= value3;
end
```

× 含有多驱动的错误代码

未初始化、多驱动均会造成仿真波形不定态X

本实验相关的代码规范

③ 组合逻辑的if必有else、case必有default，否则产生锁存器latch

- 组合逻辑中，只有latch能像寄存器一样维持电平不变
  - 但latch是电平触发，易受毛刺干扰，会造成仿真、下板行为不一致
  - 产生latch的根源：组合逻辑的信号取值维持不变，且不是常量

√
```
wire sigA = inputA & inputB;
wire [3:0] sigB = 4'b0110;
```

√
```
reg [3:0] reg_abc;
always @(*) begin
    if (flag) reg_abc = value1;
    else      reg_abc = value2;
end
```

所有可能出现的情况下，组合逻辑变量均有外部驱动，则不会出现latch

```
reg [3:0] reg_abc;
always @(*) begin
    if (flag) reg_abc = value1;
end
```

× 有latch

```
reg [3:0] reg_abc;
always @(*) begin
    if (flag) reg_abc = value1;
    else      reg_abc = reg_abc;
end
```

× 有latch

错误原因：代码逻辑是flag无效时reg_abc要维持不变

本实验相关的代码规范

③ 组合逻辑的if必有else、case必有default，否则产生锁存器latch

- 组合逻辑中，只有latch能像寄存器一样维持电平不变
  - 但latch是电平触发，易受毛刺干扰，会造成仿真、下板行为不一致
  - 产生latch的根源：组合逻辑的信号取值维持不变，且不是常量

```
reg [3:0] reg_abc;
always @(*) begin
    case (condition)
        4'h0    : reg_abc = value0;
        4'h1    : reg_abc = value1;
        4'h2    : reg_abc = value2;
        default: reg_abc = value3;
    endcase
end
```

√

```
reg [3:0] reg_abc;
always @(*) begin
    case (condition)
        4'h0 : reg_abc = value0;
        4'h1 : reg_abc = value1;
        4'h2 : reg_abc = value2;
    endcase
end
```

× 有latch

```
reg [3:0] reg_abc;
always @(*) begin
    case (condition)
        4'h0    : reg_abc = value0;
        4'h1    : reg_abc = value1;
        4'h2    : reg_abc = value2;
        default: reg_abc = reg_abc;
    endcase
end
```

× 有latch

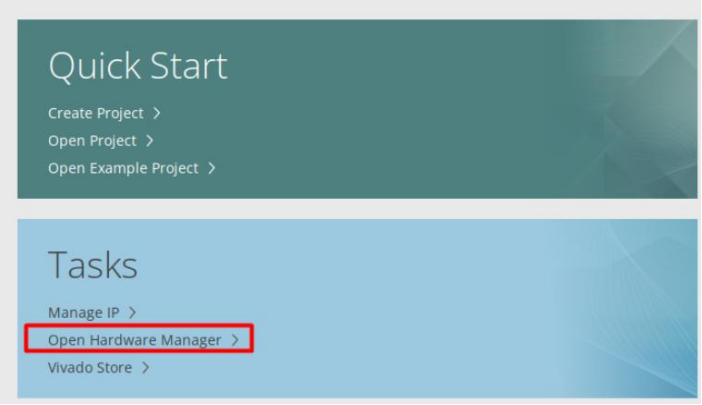
实验步骤

- 创建工程，工程名为flowing_water_lights；
- 编写并添加设计文件flowing_water_lights.v；
- 添加提供的仿真文件testbench.v，并完成仿真；
- 编写并添加约束文件，并综合实现，生成比特流；
- 将生成的比特流下载到开发板验证。

课上检查&课后提交

- 课上检查：
  - 流水灯开发板验证通过：启停、频率切换、方向切换、模式切换
  - 检查截止时间：**下周的实验课**
- 课后提交：请参照在线指导书的要求

- 本次实验课没做完，可于下次课找本班老师检查
- 备份好比特流文件，下次可直接在Vivado首页点击“Open Hardware Manager”连接开发板烧录，无需再重新生成



开始实验